AI FOR TECHNICAL DEBT RISK QUANTIFICATION: A MULTI-SOURCE, FORECAST-AWARE FRAMEWORK FOR EVIDENCE-BASED PRIORITIZATION

Sahil Khan
School of Computer Science and Engineering, Lovely Professional University, India

Abstract Technical debt (TD) describes the future cost and constraint created when software teams choose expedient solutions over more sustainable ones. Although automated analyzers and machine-learning models can identify debt-related signals, identification alone does not quantify the risk of leaving an item unresolved. This paper synthesizes recent research on TD detection, self-admitted technical debt, principal estimation, temporal forecasting, explainability, and automated management. It then proposes a multi-source, forecast-aware framework that separates (i) the probability that a candidate is genuine debt, (ii) one-time remediation principal, (iii) future debt interest, and (iv) the probability and impact of adverse engineering or operational events. The framework combines these quantities with remediation effectiveness, business benefit, dependencies, and engineering capacity to support risk-adjusted prioritization. A synthetic example illustrates the calculation; it is not an empirical evaluation. The paper also specifies a validation protocol, metrics, governance controls, and research agenda. The central argument is that trustworthy TD risk quantification requires calibrated, explainable estimates and explicit decision context rather than a single unqualified debt score.

Index Terms Artificial intelligence, technical debt, software quality, risk quantification, machine learning, debt principal, debt interest, software maintenance, explainable AI.

I. INTRODUCTION

Modern software systems are continuously changed to meet new requirements, integrate services, satisfy security obligations, and respond to market pressure. Under delivery constraints, teams may knowingly or unknowingly accept shortcuts: duplicated logic, weak modular boundaries, deferred tests, obsolete dependencies, incomplete requirements, or fragile data pipelines. Such compromises are commonly discussed as technical debt (TD). The metaphor is useful because it distinguishes the immediate benefit of a shortcut from the future effort and constraints it can create. Yet debt is not equivalent to every static-analysis warning, and its consequences depend on context, change activity, architecture, operational criticality, and the alternatives available to a team.

A practical difficulty is that organizations often have more debt liabilities than available engineering capacity. Prioritization therefore requires more than counting code smells or summing tool-reported remediation minutes. Research on prioritization finds heterogeneous strategies and no universally agreed factor set [1]. Machine-learning work has demonstrated that repository and source-code features can identify classes with high TD [2], while studies of remediation-time estimates caution that automated estimates may not match actual repair effort [3]. Principal and interest are also related but distinct economic dimensions [4]. These observations motivate a shift from "How much debt is present?" to "What is the expected consequence of leaving this liability unresolved, and what is the value of repaying it now?"

This paper addresses that question by synthesizing AI-enabled TD research and proposing a risk-quantification framework. The contributions are: 1) a structured synthesis of recent evidence across detection, measurement, forecasting, and management; 2) a decomposition of debt confidence, principal, interest, and adverse-event exposure; 3) a multi-source AI architecture for calibrated estimates and interpretable priorities; and 4) a validation and governance protocol suitable for empirical implementation. The framework is a research proposal, not a deployed system or a claim of experimentally verified performance.

II. CONCEPTS AND PROBLEM FORMULATION

A. Technical Debt as a Set of Liabilities

TD is best represented as a set of liabilities attached to identifiable software artifacts or decisions. Relevant categories include code debt (e.g., duplicated or complex implementation), design and architecture debt (e.g., violated modularity or costly dependencies), requirements debt (e.g., ambiguity, incompleteness, or deferred validation), test debt, and operational or infrastructure debt. Self-admitted technical debt (SATD) is explicitly acknowledged in comments, commit messages, reviews, or issue trackers; latent debt is not necessarily documented and must be inferred from structural or behavioral evidence. Requirements TD requires different evidence from code TD, and its quantification has been modeled as a distinct problem [11], [18]. Embedded-system studies further show that context and domain constraints affect how practitioners recognize and manage SATD [13].

B. Principal, Interest, and Risk Are Not Synonyms

The debt principal, P^i, is the estimated one-time effort or cost to remove liability i. The interest, $I^i(H)$, is the additional effort, delay, or maintenance burden expected over a future horizon H because the liability remains. Principal is therefore a repayment cost, while interest is a forward-looking consequence. Empirical investigation of principal and interest supports examining their relationship but does not justify collapsing them into one measure [4]. Tool-based remediation estimates can be diffuse or inaccurate, reinforcing the need to calibrate predictions against actual repair observations [3].

Risk adds a further dimension: the chance that unresolved debt contributes to an adverse event and the severity of that event. Examples include security exposure, defects, release delays, outages, or blocked product change. Studies have investigated TD as an indicator of software security risk [21] and have examined links between TD indicators and vulnerable code changes [26]. These associations do not automatically establish causality; they do show why event likelihood and consequence should be modeled separately and validated in the target environment.

For a candidate liability i, the quantification problem is to estimate a vector rather than a scalar: debt confidence p^i; principal P^i; interest trajectory $I^i(H)$; event probabilities $q^{ik}(H)$;

event impacts L^{ik}; and expected remediation effectiveness a^i. The decision problem then considers these estimates alongside business benefit $B^i(H)$, dependencies, and limited capacity. This formulation makes assumptions visible and enables stakeholders to challenge individual inputs.

III. RELATED WORK AND EVIDENCE SYNTHESIS

Review scope and method: This paper uses a targeted narrative synthesis of 30 peer-reviewed journal and conference publications published from 2020 through November 2026, emphasizing research from 2021–2026. Searches and backward/forward citation checks focused on IEEE Xplore, ACM Digital Library, SpringerLink, ScienceDirect, Wiley, and publisher or institutional records. The selection prioritizes studies directly addressing TD identification, SATD mining, quantification, forecasting, explainability, risk, and automation. It is not claimed to be an exhaustive systematic review: no PRISMA-style screening count or exhaustive corpus is asserted. The purpose is to connect related research streams that often use different outcome variables and evaluation procedures.

A. Automated Identification and SATD Mining

Machine learning has been applied to identify high-TD software classes using code, repository, and development-process features. Tsoukalas et al. report a classifier-based approach evaluated across Java projects, demonstrating that predictive identification can reduce the inspection search space [2]. However, the output is a high-debt classification, not a direct estimate of monetary exposure or organizational risk. SATD research extends the evidence base beyond source code: issue descriptions, commit messages, pull requests, and code reviews can contain explicit admissions that static analyzers cannot observe [6], [7], [12]. Li et al. show that issue-tracking data provides a complementary source for SATD identification [6], and their multi-source study combines four natural-language sources [12].

Language models and deep-learning methods have been used to classify SATD in domain-specific repositories, including R packages [8]. SATDBailiff addresses the lifecycle problem by mining and tracking SATD rather than treating every comment as a static one-off label [9]. Conditional-statement debt research illustrates a complementary challenge: some liabilities are latent and require structural interpretation even when developers have not admitted them [10]. Code-review evidence also matters because debt-related comments can arise during review and influence review outcomes [7]. Taken together, the literature supports multi-source detection but also exposes dataset, label, and context dependence.

Recent work on architecture debt combines active learning and explainable AI to reduce expensive labeling. Sutoyo et al. report an F1-score of 0.72 for their best active-learning strategy alongside a 49% reduction in annotation effort, and use SHAP and LIME to explain predictions [30]. These results are promising for data-efficient detection; they should not be interpreted as a universal accuracy guarantee across languages, organizations, or debt types.

B. Quantification, Principal, and Architectural Debt

Quantification research reveals that different approaches often measure different objects: detected smells, remediation effort, quality gaps, or return on refactoring. A systematic mapping study and conceptual model emphasize the need to make the target construct explicit before comparing scores [17]. Architectural debt indices have used machine learning and architectural-smell severity to estimate principal at a level that is actionable for architects [14]. Design-rule theory provides another route for relating design constraints to debt management [15]. Research on requirements debt similarly shows that quantification must account for requirement artifacts and their lifecycle rather than simply reuse code-smell metrics [11], [18].

The automation landscape remains fragmented. A state-of-the-art mapping of TD management automation identifies tools and opportunities but also highlights gaps in integration and end-to-end automation [16]. Work on automated repayment, including generating code intended to remove SATD, is an emerging direction [19]; generated patches still require tests, review, and context-specific verification. Studies of maintainability also suggest that maintainable code can yield benefits through fewer defects and shorter implementation time, supporting a broader view of repayment value [20].

C. Forecasting, Prioritization, and AI-Enabled Systems

A debt snapshot is insufficient when liabilities evolve with change activity. Temporal convolutional networks have been applied to just-in-time forecasting of debt principal [5]. Forecasting has also been compared with multivariate time-series approaches using the SQALE index [22], while recent research considers time dependence and seasonal effects in code-debt prediction [27]. At class level, forecasted evolution and change proneness have been incorporated into prioritization to identify components at risk of becoming difficult to maintain [1], [22]. These studies support forecasting as a decision input, not as a substitute for principal estimation or business-impact analysis.

Debt also changes as architectures and delivery models change. Research on microservices traces debt evolution across service-based systems [25]. AI-enabled systems introduce additional debt in data, infrastructure, pipelines, model integration, and governance, beyond conventional source code [24]. A recent multivocal review traces the evolution of TD discussions from DevOps to generative AI, reinforcing the need to expand the monitored evidence surface [29]. Machine-learning surveys also report that model choice is context-dependent; no single classifier dominates all TD and SATD identification settings [28].

Explainability is central to operational adoption. Local and global explainability methods have been examined for TD identification [23], while active-learning work extends explainability to architectural-debt labels [30]. A system that merely ranks artifacts without showing the evidence, confidence, and uncertainty behind the ranking risks turning model output into an unchallengeable authority. The proposed framework therefore includes explanations and human review as core components.

Across these streams, three methodological patterns recur. First, labels are expensive and ambiguous: a SATD phrase can be historical, aspirational, or already resolved, while an architectural smell may require expert interpretation. Second, data are strongly imbalanced because actionable debt items are relatively sparse compared with ordinary code and issue text. Third, model performance is sensitive to granularity and context. A class-level code-smell classifier cannot be assumed to generalize to requirements, operations, or machine-learning pipelines. Recent work on active learning and multi-source

SATD extraction is useful because it treats annotation and source coverage as part of the technical solution rather than as incidental preprocessing [12], [28], [30].

TABLE I. EVIDENCE SYNTHESIS: RESEARCH STREAMS AND IMPLICATIONS

Research stream	Representative evidence	Implication for risk quantification
Detection / SATD	ML classifiers [2]; issue and multi-source NLP [6], [12]; SATD tracking [9]	Fuse code and natural-language evidence; retain debt-type labels.
Principal / architecture	Remediation-estimate study [3]; ML architecture index [14]; design rules [15]	Calibrate effort estimates; quantify at actionable artifact level.
Forecasting / priority	TCN principal forecast [5]; SQALE comparison [22]; seasonal models [27]	Forecast debt trajectory and change exposure over a stated horizon.
Risk / domain context	Security-risk ML [21]; AI-enabled TD [24]; microservices [25]	Use domain-specific event likelihood and impact; broaden data sources.
Trust / governance	XAI [23], active learning [30], automation mapping [16]	Explain decisions, budget labeling effort, and preserve human approval.

D. Research Gap

The literature has advanced each stage, but a unified, operationally interpretable risk model remains under-specified. Detection papers often classify debt presence; quantification papers estimate principal or index values; forecasting papers predict future debt measures; and security studies predict risk-related outcomes. These outputs answer different questions. The gap is not the absence of AI classifiers; it is the lack of a transparent bridge from heterogeneous evidence to a calibrated, horizon-specific expected exposure and a capacity-aware repayment decision. The framework below addresses that integration gap while preserving uncertainty and avoiding unjustified causal claims.

IV. PROPOSED MULTI-SOURCE AI FRAMEWORK

The proposed architecture has six layers: evidence ingestion, artifact normalization, debt identification, multi-head estimation, risk aggregation, and decision governance (Fig. 1). It is designed to operate at class, component, service, requirement, or pipeline level, provided that each unit is assigned a stable identifier and its evidence is time-stamped. The model should retain provenance from every prediction to the source commit, issue, code metric, dependency, test result, or operational record that contributed to it.

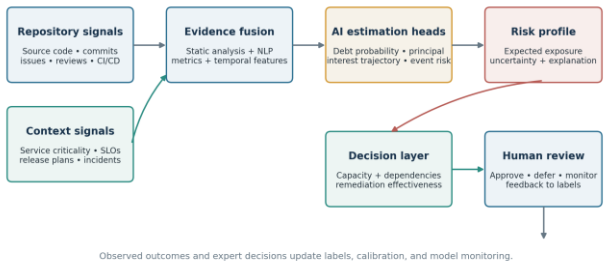


Fig. 1. Proposed multi-source architecture for AI-assisted technical-debt risk quantification.

A. Evidence Ingestion and Representation

Static evidence includes complexity, duplication, dependency structure, code churn, test coverage, architectural smells, build warnings, and dependency age. Repository evidence includes commits, pull requests, review comments, issue histories, and ownership patterns. Operational evidence includes incident records, defect reports, deployment failures, service-level objectives, vulnerability findings, and change lead time. Contextual evidence includes component criticality, regulatory obligations, customer exposure, release commitments, and planned modernization. Natural-language artifacts should be parsed with domain-tuned language models or conventional NLP; code and graph features should be represented separately before late fusion. A single unstructured embedding is unlikely to preserve the distinct meanings of debt type, repair effort, and business consequence.

Every observation should carry a timestamp, repository and artifact identifiers, collection method, source confidence, and version. This enables temporal alignment and prevents information created after the prediction cutoff from leaking into training. For organizations with limited labeled examples, active learning can prioritize cases where model uncertainty or stakeholder disagreement is high [30].

A1. Artifact Identity and Temporal Alignment

A key implementation problem is joining heterogeneous evidence to the same liability. A code smell may be attached to a method, a dependency issue to a package, an incident to a service, and a requirement gap to an epic. The ingestion layer should maintain a typed artifact graph whose nodes represent classes, modules, services, requirements, data assets, models, and deployment pipelines. Edges represent calls, imports, ownership, data flow, requirement traceability, and deployment dependency. Each item should be versioned so that features are computed using the repository state available at prediction time. When exact linkage is unavailable, the system should record a probabilistic link rather than silently merge records.

Temporal alignment also prevents a common evaluation error: using a remediation commit, post-incident ticket, or later code metric as a predictor for a decision that supposedly occurred earlier. A feature store should therefore retain event time and ingestion time, with a strict “as-of” query for model training. Data quality checks should flag missing histories, renamed modules, deleted artifacts, and changes in static-analysis configuration. Without this discipline, a model may learn repository bookkeeping rather than future risk.

B. Debt Detection and Type Classification

For candidate i, the detection head estimates $p^i = P(D^i=1 \mid X^i)$, where D^i indicates whether the candidate is a genuine debt liability and X^i is its evidence. A hierarchical classifier should first distinguish debt from non-debt, then predict type (code, design, architecture, requirements, test, infrastructure/data, or AI/model debt). SATD labels should be treated as evidence, not ground truth: a comment such as “temporary workaround” can be stale, while undocumented architecture debt may be substantial. Expert adjudication and multiple evidence sources are therefore required. Model outputs should include calibrated probability, label provenance, and an uncertainty flag.

A practical classifier can use a cascade: a high-recall candidate generator based on rules and keyword patterns, followed by a supervised classifier that uses code, text, and graph context, and finally an expert review queue for uncertain or high-impact predictions. Thresholds should be selected by the cost of false positives and false negatives, not by defaulting to 0.50. For example, a security-critical module may justify a lower alert threshold if review capacity is available, whereas a low-

impact package may use a higher threshold. Class probabilities should be calibrated on a held-out set using methods such as isotonic regression or Platt scaling, and the system should abstain when the evidence falls outside the training distribution.

C. Principal and Interest Estimation

The principal head estimates P^i in a measurable unit such as engineer-hours, person-days, or fully loaded currency. It should be trained against observed remediation logs, not only vendor-provided remediation minutes. Candidate predictors include debt category, affected lines or modules, dependency depth, historical repair duration for similar items, test availability, code ownership dispersion, and required coordination. Since effort distributions are often skewed, quantile regression or probabilistic regression is preferable to a single point estimate. Predictions should report a median and interval (e.g., P50 and P80) so that planning decisions can account for estimation uncertainty.

The interest head estimates future extra effort caused by retaining the item over a horizon H. A practical operational proxy is the incremental effort associated with changes to the affected artifact relative to a comparable low-debt baseline, adjusted for change frequency and dependency exposure. Time-series models, temporal convolutional networks, and gradient-boosted lag-feature models can be compared [5], [22], [27]. The target should be explicitly defined: additional change effort, maintenance rework, forecasted principal growth, or another observable outcome. These targets are related but not interchangeable.

Principal prediction should distinguish implementation effort from elapsed calendar time. A 20-hour repair may require several calendar weeks if it depends on an external vendor or a scarce architect. For this reason, the system can provide separate estimates for engineering effort, elapsed lead time, and coordination burden. Where historical repair records are sparse, hierarchical models can partially pool information across related teams or artifact types while preserving team-level variation. The resulting estimates should be benchmarked against simple category medians so that a complex model must demonstrate incremental value.

Interest estimation is more difficult because the counterfactual—what the same component would have cost if the debt had not existed—is unobserved. A practical proxy may compare changes to affected components before and after repayment, use matched components with similar size and change frequency, or estimate the excess effort associated with debt indicators in a longitudinal panel. Each proxy has assumptions. A before/after comparison can be confounded by simultaneous rewrites, staffing changes, or product growth. The paper therefore recommends reporting the proxy definition and sensitivity analysis alongside every interest estimate.

D. Event Likelihood and Consequence

For each event type k (e.g., vulnerability, production defect, service interruption, release delay), the event head estimates q_ik(H), the probability that unresolved liability i contributes to that event within horizon H. A separate impact model estimates L_ik, the consequence conditional on the event, in a consistent unit such as monetary loss, engineer-hours, or a normalized business-impact scale. Domain-specific predictors may include exploitability, attack surface, service criticality, incident history, deployment frequency, and dependency centrality. Security-risk studies provide a rationale for testing these links [21], [26], but observed associations must not be described as causal effects without an identification strategy.

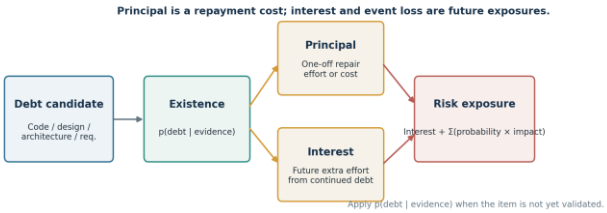


Fig. 2. Separating debt existence, repayment principal, future interest, and event exposure.

Event models should avoid treating all defects or vulnerabilities as consequences of a particular debt item. Attribution can be represented at different strengths: directly linked (the event cites the artifact), plausibly contributing (the artifact is in the causal path), or merely correlated (the artifact shares a statistical pattern). The strongest class can support supervised attribution labels; weaker classes should be used cautiously and subjected to sensitivity analysis. Impacts should include both direct cost and operational duration where appropriate, but duplicate components of the same loss must not be counted twice.

E. Risk Aggregation and Prioritization

The risk exposure of liability i over horizon H is defined as:

$$RE^i(H) = p^i \times [\, I^i(H) + \Sigma^i q^{ik}(H) \times L^{ik} \,] \qquad (1)$$

For an already validated debt item, p_i may be set to 1. The probability term prevents uncertain candidates from being treated as confirmed liabilities. Interest and event impacts must be converted to a common unit before addition. Principal is not added to exposure because it is the one-time cost of repayment rather than the cost of retaining the debt; it enters the decision stage:

$$NB^i(H) = a^i \times RE^i(H) + B^i(H) - P^i \qquad (2)$$

Here, a^i is the expected fraction of exposure removed by the proposed remediation, $B^i(H)$ is the additional product or strategic benefit unlocked by repayment, and NB^i is expected net benefit. A simple efficiency ratio is NB^i/P^i, but the ratio should not replace constrained portfolio optimization. Given engineering capacity C, a more complete selection problem maximizes the sum of expected net benefits over selected liabilities, subject to total repair effort not exceeding C and to dependency constraints. This formulation also exposes a critical assumption: the same remediation may not eliminate all associated risk, so a^i must be estimated or elicited rather than silently assumed to equal one.

The output should be a risk profile containing p^i, P^i, $I^i(H)$, each q^{ik} and L^{ik}, uncertainty intervals, a^i, NB^i, and explanation features. This prevents teams from mistaking a high-principal item for a high-risk item or prioritizing a low-confidence classifier output as if it were an audited liability.

F. Portfolio Selection and Dependency Constraints

A team rarely chooses one item in isolation. If x_i is 1 when liability i is selected for remediation and 0 otherwise, a simplified portfolio problem is:

$$\text{maximize } \Sigma^i x^i NB^i \quad \text{subject to } \Sigma^i x^i P^i \le C \qquad (3)$$

where C is the available engineering capacity for the planning period. If remediation of item i requires item j to be completed

first, the constraint $x^i \leq x^j$ can represent that dependency. In practice, components may share the same root cause or risk pathway; simply summing their exposures can double-count benefits. A portfolio model should therefore include dependency graphs, overlapping incident scenarios, or a scenario-based simulation that estimates joint outcomes. The mathematical program is a conceptual decision layer; a real deployment may need integer optimization, heuristics, or a human-reviewed shortlist when estimates are uncertain.

G. Uncertainty and Sensitivity

Point estimates can create false precision. The framework should preserve predictive distributions for principal, interest, event probability, and impact. Monte Carlo simulation can sample from these distributions to estimate the probability that net benefit is positive, the range of portfolio outcomes, and the chance that capacity will be exceeded. Sensitivity analysis should vary the forecast horizon, incident base rate, remediation effectiveness, discount rate (if monetary values are used), and impact assumptions. When a ranking changes under small perturbations, the system should mark the items as a decision tie and request expert review rather than display a misleadingly precise rank.

V. FEATURE MODEL AND IMPLEMENTATION CHOICES

TABLE II. FEATURE FAMILIES AND PREDICTION TARGETS

Feature family	Example variables	Primary target
Code / design	Complexity, duplication, modularity violations, dependency depth	Debt type; principal
Repository / process	Churn, change frequency, co-change, ownership dispersion, review comments	Debt confidence; interest
Language artifacts	SATD phrases, issue topics, commit and review semantics	Debt confidence; category
Quality / security	Defects, vulnerabilities, test gaps, static-analysis history	Event likelihood
Operations / business	Criticality, incidents, SLOs, release delay, customer exposure	Impact; benefit; priority

A. Model Selection by Prediction Target

Model choice should follow the prediction target and available labels. Gradient-boosted trees and random forests are strong tabular baselines; logistic regression provides a transparent reference. Transformer encoders can represent comments and issue text, while graph models may capture dependency structure. Temporal models should be compared with simple autoregressive and seasonal baselines rather than assumed superior. The literature reports diverse approaches and contexts [2], [5], [8], [22], [27], [28]. For sparse or noisy labels, weak supervision and active learning can reduce annotation burden, but the labeling protocol and disagreement rate should be documented.

B. Modular Prediction Heads and Calibration

A production system should preserve modularity between model heads. A single end-to-end neural network may predict several outputs, but separate heads make it easier to audit whether a model is good at detecting debt yet poor at estimating repair effort. Calibration should be evaluated separately for each head, and the aggregation layer should reject incompatible units, missing horizons, or stale context. An organization should also be able to replace one model (for example, the event-risk model) without rebuilding the entire pipeline.

C. Missing Data, Drift, and Model Monitoring

Missingness is informative in software repositories: teams with weak review practices may also have incomplete issue records, and legacy systems may lack consistent telemetry. Missing values should be represented with explicit indicators and audited by project and debt type. Imputation should be learned only on training data. Model monitoring should track feature drift, label delay, calibration decay, and changes in static-analysis rules. A model retrained after a major repository migration should be treated as a new version, with benchmark results and a rollback path.

D. Explainability and Decision Records

An explanation should answer four separate questions: why was this artifact flagged as debt; which evidence drove the principal estimate; what signals increased the forecasted interest or event exposure; and which assumptions made repayment valuable. Local explanations may show the main features for an individual item, while global explanations reveal systematic model behavior across a portfolio. For text-derived predictions, the system should show relevant comment or issue spans with access controls. Every ranking decision should be stored with model version, feature snapshot, confidence interval, reviewer action, and reason for override. This creates an audit trail and supports later evaluation of whether recommendations changed engineering outcomes.

VI. ILLUSTRATIVE QUANTIFICATION EXAMPLE

The following example uses synthetic values solely to demonstrate the equations; it is not a dataset, experiment, or claim about observed software systems. Consider two liabilities over a 12-month horizon. To keep units consistent, all effort and event impacts are expressed in engineering-hour equivalents. Liability A is a shared authentication module with elevated security significance. Liability B is duplicate report-generation logic with lower operational criticality. For illustration, the interest term is forecast from expected additional maintenance effort, and the event term is probability multiplied by conditional impact.

TABLE III. SYNTHETIC EXAMPLE OF RISK-ADJUSTED PRIORITIZATION

Variable	Liability A	Liability B
Debt probability, p_i	0.92	0.95
Principal, P^i	18 h	10 h
Interest over 12 months, I^i	24 h	12 h
Event probability × impact	0.10 × 160 = 16 h	0.04 × 25 = 1 h
Risk exposure, RE^i	0.92 × (24 + 16) = 36.8 h	0.95 × (12 + 1) = 12.35 h
Remediation effectiveness, a_i	0.85	0.90
Business benefit, B^i	0 h	0 h
Net benefit, NB^i	0.85 × 36.8 − 18 = 13.28 h	0.90 × 12.35 − 10 = 1.12 h
NB^i / P^i	0.74	0.11

Liability A has the larger principal, yet its forecasted exposure and expected net benefit are also substantially higher. The example demonstrates why sorting only by remediation cost, code-smell count, or principal can produce a poor order. The ranking would change if the forecast horizon, event assumptions, remediation effectiveness, or business benefit changed. In an actual deployment, stakeholders should examine intervals and sensitivity analyses rather than accepting point estimates as exact.

Sensitivity check: suppose the estimated event probability for Liability A falls from 0.10 to 0.03 while all other synthetic inputs remain fixed. Its event contribution decreases from 16 to 4.8 effort-equivalent hours; risk exposure becomes $0.92 \times (24 + 4.8) = 26.50$ hours, and net benefit becomes $0.85 \times 26.50 - 18 = 4.53$ hours. Liability A remains positive-value in this scenario but its margin is materially smaller. If remediation effectiveness were also reduced, the priority could reverse. This sensitivity check illustrates why organizations should report plausible ranges and assumptions instead of presenting an isolated ranking as an objective fact.

VII. VALIDATION AND EXPERIMENTAL DESIGN

A credible evaluation should compare the proposed framework with interpretable baselines and assess each prediction head independently before measuring decision utility. Data should be split chronologically and by project or organization: random row-level splitting can place nearly identical artifacts or later commits in both training and test sets, inflating performance. The evaluation protocol should define a prediction cutoff, ensure every feature existed at that time, and reserve at least one project or business unit for external validation. The framework should be evaluated at the same granularity at which managers act (e.g., class, service, requirement, or pipeline).

Labels should be constructed from multiple sources. Debt existence and category require expert adjudication with an explicit rubric; principal should be measured from actual remediation records; interest should use a defined counterfactual proxy or longitudinal outcome; and event outcomes should be linked through incident, vulnerability, or defect records. Annotators should record confidence and disagreements. Where possible, a sample should be double-labelled and adjudicated. SATD comments should not automatically be labeled as actionable debt without considering whether the compromise remains present [6], [9], [12].

Detection should be reported with precision, recall, F1, Matthews correlation coefficient, and precision-recall area under the curve; ranking should additionally report recall at k and the proportion of confirmed high-impact liabilities in the top-k list. Probability calibration should use Brier score, reliability diagrams, and expected calibration error. Principal estimates should be evaluated with mean absolute error, median absolute error, and quantile loss. Forecasts should use time-aware MAE or MASE and interval coverage. Prioritization should be evaluated by cost-weighted recall, NDCG@k, and realized or simulated net benefit under a fixed engineering-hour budget. No single metric is sufficient.

Baselines should include static-analysis rules, logistic regression, random forest or gradient boosting, a transformer-based SATD model, and simple univariate or seasonal forecasting. Ablation tests should remove language artifacts, architecture context, operational events, and calibration in turn. Cross-language, cross-project, and cross-team transfer tests are necessary to estimate generalization. Performance should be stratified by debt type and component criticality; a model that performs well on code smells but poorly on requirements or infrastructure debt should not be presented as a general TD risk model.

TABLE IV. EVALUATION TARGETS AND METRICS BY MODEL HEAD

Model head	Observed target	Core evaluation
Debt detection	Expert-adjudicated liability and debt type	PR-AUC, recall@k, F1, MCC, calibration
Principal	Actual remediation effort / cost	MAE, median AE, P80 quantile loss
Interest forecast	Predefined future extra-effort proxy	MASE, horizon MAE, interval coverage
Event risk	Linked future incident / vulnerability	Brier score, calibration, cost-weighted recall
Priority policy	Observed or simulated portfolio outcomes	NDCG@k, net benefit, capacity utilization

Evaluation should distinguish predictive validity from decision validity. A model may slightly improve F1 yet fail to improve the quality of the top-ranked remediation set. Conversely, a calibrated risk model may deliver useful decision support even when its raw detection score is not the highest, because managers can make informed trade-offs using confidence intervals and event impacts. A prospective shadow evaluation is therefore desirable: generate recommendations without exposing them to delivery teams, then compare predictions with subsequent repair effort, incidents, and change outcomes. Afterward, a controlled rollout can test whether exposing the risk profile changes allocation, lead time, defects, or operational reliability.

For ranking comparisons, evaluation should use a common budget and common candidate pool. A principal-only baseline, a static-analysis severity ranking, a change-proneness ranking, and a random or business-as-usual selection should be compared with the proposed risk-adjusted policy. Report both the number of completed items and the realized reduction in exposure, not only the score of items selected. Where teams have different staffing or release calendars, normalize capacity and report uncertainty across repeated time windows. Bootstrap confidence intervals or hierarchical project-level models can quantify between-project variation, and multiple-comparison corrections should be considered when many debt types or metrics are tested.

A particularly important design issue is delayed feedback. Repair outcomes may take months to observe, incidents can be rare, and the absence of an incident does not prove that a risk estimate was wrong. Researchers should use survival analysis or censoring-aware methods when event observation windows differ. When remediation changes the underlying system, post-treatment measurements should not be used as if they were untreated controls. A clear evaluation protocol will state the observation window, censoring rules, and treatment of items that were deferred, split, or superseded.

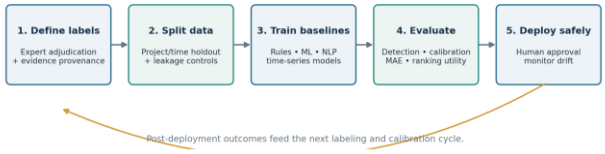


Fig. 3. Recommended validation and deployment feedback loop.

A. Research Questions for Empirical Evaluation

An empirical study of the framework can be organized around five research questions: (RQ1) Does multi-source evidence improve debt detection and calibration over source-specific

models? (RQ2) Can historical remediation records predict principal more accurately than static tool estimates? (RQ3) Do temporal models improve interest forecasting over non-temporal baselines across projects and debt categories? (RQ4) Does adding event probability and impact improve top-k prioritization utility compared with principal-only and smell-count rankings? (RQ5) Do local explanations and uncertainty intervals improve reviewer agreement, appropriate override rates, and trust? These questions make the proposed contribution falsifiable and prevent detection accuracy from being used as a proxy for risk-quantification quality.

B. Staged Adoption Roadmap

A low-risk implementation can proceed in stages. First, establish a debt taxonomy, artifact identifiers, and a data dictionary. Second, deploy a candidate-detection model in shadow mode, allowing engineers to validate labels without changing sprint priorities. Third, collect actual remediation effort and forecast targets, then calibrate principal and interest models. Fourth, integrate security, defect, and operational event data where linkage is defensible. Fifth, introduce a decision-support dashboard that shows the full risk profile and supports human overrides. Only after prospective evaluation should the organization consider portfolio optimization or automated repair suggestions. This incremental approach reduces the chance that immature predictions influence critical release decisions.

VIII. GOVERNANCE, THREATS, AND LIMITATIONS

Construct validity is the first threat: a code smell, a SATD comment, a vendor remediation estimate, and a production incident are not interchangeable measurements of debt. The target definition and unit must be published with every result. Internal validity is threatened by confounding: high-churn components may both accumulate debt and receive more bug reports, without debt alone causing the incidents. Causal claims require stronger designs, such as matched comparisons, natural experiments, or carefully specified longitudinal models. External validity is limited when training data comes from open-source Java projects but deployment involves regulated embedded software, microservices, or AI pipelines. Data leakage and temporal drift can also make apparently high model performance misleading.

Explainability should be tied to evidence stakeholders can inspect: top contributing code metrics, relevant issue or review excerpts, affected dependencies, and changes in forecast trajectory. SHAP or LIME outputs can help explain model behavior, but they do not prove that a feature causes debt or an incident [23], [30]. Model cards should document training population, label policy, intended use, excluded debt types, performance by subgroup, known failure modes, and recalibration triggers. Teams should retain a manual override and an appeal path.

Privacy and security matter because repository data may contain credentials, customer identifiers, proprietary code, or vulnerability details. Ingestion should follow least-privilege access, secret scanning, retention limits, and controlled model-hosting policies. Generated remediation patches should be isolated, tested, reviewed, and threat-modeled before merge. Automated repayment is promising [19], but autonomous changes to security-critical or safety-critical components should not be approved solely by a model score. Human ownership and accountability remain necessary.

Finally, a financial expected-value model can understate strategic considerations such as regulatory deadlines, architectural option value, end-of-life technology, or knowledge concentration. Such factors should enter as explicit constraints or documented benefit terms rather than hidden multipliers. AI-enabled products need additional monitoring for data, model, pipeline, and infrastructure debt [24], [29]. The framework is therefore a decision-support mechanism, not a replacement for architecture governance or engineering judgment.

Operational governance should define who owns each model head and who can approve remediation. Engineering leaders may own capacity and prioritization policy; software maintainers validate debt labels; security specialists assess vulnerability consequence; product owners estimate business benefit; and model-risk or architecture governance reviews data access and evaluation. Decision thresholds should be reviewed when staffing, product criticality, or release cadence changes. A clear responsibility matrix is especially important when a platform team publishes risk scores that influence multiple product teams.

Further limitations concern fairness and incentives. Teams with better telemetry can appear more indebted simply because more liabilities are visible, while teams with poor observability may be under-scored. If a risk score is used as a performance metric for individual developers, it can encourage under-reporting of SATD or avoidance of difficult modules. Scores should therefore be used to prioritize system improvement, not to rank employees. A balanced evaluation should include coverage and false-negative analysis across legacy/new systems, languages, teams, and component criticality.

C. Threshold Policy and Organizational Risk Appetite

Detection and prioritization thresholds are policy choices, not fixed properties of a classifier. A low debt-probability threshold increases recall but also increases review workload; a high threshold reduces workload but may hide liabilities in critical components. The appropriate threshold depends on the cost of missing a liability, the cost of reviewing a false positive, and available remediation capacity. For this reason, the system should report precision-recall curves and allow role-specific thresholds while preserving a consistent underlying probability estimate. A security team may choose to inspect a broader set of potentially vulnerable components, whereas a feature team may prioritize a smaller list that can realistically be repaired in the next iteration.

Risk appetite should also be expressed separately from model output. One organization may accept a higher likelihood of minor defects to preserve release velocity; another may require near-zero tolerance for vulnerabilities in a regulated service. The same calibrated risk profile can support both policies if the event categories and impacts are explicit. The decision layer should therefore distinguish the estimated state of the software from the policy applied to it. This separation reduces the risk that managers interpret a score as an objective mandate when it is partly a reflection of preferences, constraints, and contractual obligations.

The economics of false negatives and false positives should be measured after deployment. A false positive can consume scarce engineering time, interrupt feature work, or encourage unnecessary refactoring. A false negative can allow fragile components to remain in place until they contribute to expensive rework or an incident. These costs are asymmetric and vary by

component criticality. An operational study should track not only classifier confusion matrices but also review time, accepted-versus-rejected recommendations, remediation completion, and subsequent outcomes. Thresholds can then be tuned to minimize total decision cost under a stated capacity budget, rather than maximizing a generic F1 score.

Risk scores should not be compared across teams unless measurement conditions are aligned. Differences in repository size, monitoring coverage, incident reporting, architecture style, and staffing can change both the visibility of debt and the cost of repayment. A normalized score may support within-team ranking, but cross-team comparisons require an explicit normalization model and uncertainty estimates. In particular, a team with stronger incident reporting may appear riskier than a team with weak observability. Governance should treat this as a measurement-quality issue and avoid using raw risk scores for league tables or employee performance evaluation.

IX. DISCUSSION AND RESEARCH AGENDA

The proposed framework changes the managerial question from "Which component has the most debt?" to "Which feasible repayment action is expected to reduce the most relevant future exposure?" This reframing matters because a technically severe issue may be dormant in a stable component, while a modest architectural compromise in a frequently changed service can accumulate substantial interest. It also explains why the same liability may be ranked differently by a product team, a security team, and a platform team: their event definitions, impact scales, and planning horizons differ. The model should expose these preferences explicitly rather than encode them in undocumented feature weights.

A single universal risk index is unlikely to be appropriate across organizations. Financially mature organizations may express impact in currency; early-stage teams may use engineer-hours or ordinal impact categories. Safety-critical or regulated systems may impose hard constraints that cannot be offset by expected monetary benefit. For these contexts, the portfolio layer should support non-compensatory rules, such as mandatory repayment before a release, maximum vulnerability exposure, or minimum test coverage. Weighted scores are useful for ranking discretionary work, but they should not override legal, safety, or contractual obligations.

The framework also has implications for the research community. Datasets should include more than source code and comments: issue histories, architectural models, deployment events, remediation records, and business context are needed to test risk hypotheses. Dataset documentation should describe the unit of analysis, debt taxonomy, label source, time period, language, repository governance, and missingness. Shared benchmark tasks should separate detection, principal, interest, event attribution, and prioritization so that improvements in one task are not mistaken for progress in all tasks. This separation would make comparisons more interpretable and reduce the temptation to report one headline metric for a multi-stage problem.

Several open research directions follow. First, causal estimation is needed to distinguish debt-associated incidents from incidents that debt actually contributes to. Second, counterfactual effort models should estimate how maintenance cost would differ after repayment, including changes in team productivity and feature delivery. Third, graph-based models may help quantify propagation across services, packages, requirements, and data pipelines, but their explanations must remain reviewable. Fourth, uncertainty-aware active learning could prioritize both new labels and the next liabilities requiring expert review. Fifth, generative-AI-assisted repayment should be evaluated jointly for patch correctness, security regression, maintainability, and long-term interest reduction rather than for syntactic acceptance alone [19], [29].

Finally, the value of AI should be evaluated in terms of better engineering decisions, not merely automation volume. A model that flags thousands of items but cannot explain false positives, estimate repair effort, or demonstrate improved outcomes may increase alert fatigue. A smaller calibrated system with clear provenance, explicit uncertainty, and a realistic decision budget may be more valuable. This is why human review, prospective evaluation, and transparent policy settings are not implementation details; they are part of the scientific definition of a trustworthy risk-quantification system.

Risk aggregation also requires careful treatment of time. A 12-month horizon is useful for quarterly planning, while a three-year horizon may be relevant for architectural modernization or platform replacement. Forecasts should not extrapolate linearly without checking whether the system is approaching a release freeze, a technology end-of-life date, or a planned migration. If monetary discounting is applied, the discount rate should reflect the organization's planning convention and should be stated explicitly. In a safety or security context, low-probability catastrophic events may require tail-risk measures or hard constraints rather than expected value alone. A risk dashboard should therefore support multiple horizons and show both mean exposure and severe-but-plausible scenarios.

Dependencies create another aggregation challenge. A shared library can expose several services to one underlying defect or design limitation. Adding the risk estimates of each service independently may count the same outage scenario several times. Conversely, prioritizing only the shared library may ignore service-specific mitigations that reduce exposure sooner. A graph-based representation can identify common ancestors, shared dependencies, and remediation prerequisites. Portfolio analysis can then compare alternative actions: repair the shared root cause, introduce a local containment control, or combine both. This is a natural setting for scenario analysis because the value of one repair depends on which other repairs and mitigations are selected.

The framework should also separate risk reduction from risk transfer. Replacing a local workaround with a third-party service may reduce code complexity but introduce vendor dependency, data residency, availability, or licensing risk. Similarly, generated code may remove a visible smell while increasing hidden model, provenance, or security debt. A remediation plan should therefore describe the post-change architecture and reassess the affected debt categories. The system can support this by comparing predicted before-and-after profiles, but the post-change estimate must be validated with actual deployment and maintenance outcomes rather than assumed from the patch itself.

A mature implementation may use a two-stage decision process. The first stage screens candidates with calibrated detection and risk thresholds, preserving high-recall review queues for high-impact modules. The second stage presents a short list with principal ranges, forecast scenarios, expected net

benefit, dependencies, and evidence provenance. Stakeholders can then apply explicit policy overrides, such as mandatory security remediation, contractual deadlines, or architectural sequencing. The audit record should capture whether the decision was accepted, deferred, rejected, or replaced by a mitigation, together with the reason. These outcomes are valuable labels for later evaluation but should not be fed back as unbiased ground truth: a rejected item may still be real debt, and a deferred item may remain dangerous.

This separation also provides an ethical boundary for AI-supported engineering management. Risk scores are indicators of system conditions, not measures of individual developer quality or productivity. The presence of debt may reflect inherited architecture, organizational constraints, or an intentional short-term trade-off. Teams should be encouraged to record compromises honestly without fear that SATD counts will be used punitively. Transparent governance and non-punitive reporting improve the quality of the data on which the models depend, creating a reinforcing cycle between engineering practice and measurement validity.

For operational use, each model output should follow a versioned score contract. At minimum, the contract should identify the artifact and version, prediction time, debt category, debt probability, principal unit and quantiles, interest horizon and target definition, event taxonomy, conditional impacts, remediation effectiveness assumption, calibration set, and model version. A score without a horizon or unit should be considered incomplete. This contract enables dashboards, ticketing systems, and portfolio tools to consume risk information without silently interpreting one model's output as another model's target.

Refresh cadence should follow the rate at which the underlying evidence changes. Static code and dependency indicators can be updated on each main-branch build; issue, review, and commit features can be updated with repository events; incident and vulnerability data may arrive with delay; and business criticality may change only during planning or architecture reviews. The system should retain the last-known timestamp for every source and display stale-data warnings. Model retraining should be event-driven only when enough new labels or measurable drift justify it; continuous retraining without evaluation can destabilize ranking and make decisions difficult to reproduce.

The user interface should prioritize traceability over visual complexity. A liability card can show a compact risk summary, but the reviewer should be able to expand the record to inspect the evidence, confidence interval, similar historical repairs, affected dependants, and forecast assumptions. Items with high expected exposure but low debt confidence should be separated from confirmed liabilities requiring remediation. Likewise, a high-principal, low-interest item should not be visually conflated with a low-principal, high-event-risk item. This presentation helps teams select the appropriate action: repay, mitigate, monitor, gather more evidence, or accept the risk.

The framework also supports an explicit "defer" decision. Deferral is not a failure if the estimated cost of repayment exceeds the current expected benefit or if a higher-priority release obligation consumes the capacity. However, deferral should create a review date, owner, trigger condition, and re-estimation policy. Examples of triggers include a material increase in change frequency, a newly reported vulnerability, an upcoming service migration, a rise in incident severity, or a change in regulatory classification. A trigger-based review is more defensible than treating a low score as permanent evidence that the debt is harmless.

To avoid overfitting organizational policy to a single planning cycle, deployment evaluations should span multiple quarters and include both normal and stressed periods. Incident frequency can rise during a major release, while principal estimates may change when team composition changes. A useful evaluation should therefore compare performance under ordinary delivery conditions and under known transitions such as migrations, platform upgrades, or rapid product expansion. This longitudinal perspective is particularly important for AI-enabled and microservice systems, where infrastructure and integration constraints can evolve faster than source-code indicators [24], [25], [29].

Finally, operational usefulness should be assessed through the full decision chain: whether a candidate was noticed, whether the evidence was judged credible, whether the estimated repair scope was accurate, whether the selected action was completed, and whether the intended risk reduction occurred. This chain can fail at several points. A correct detector may surface an item no team owns; an accurate principal estimate may be impossible to schedule; a completed refactoring may not address the underlying incident pathway. Recording these intermediate outcomes allows researchers to distinguish model error from process friction and to improve the component of the system that actually limits value.

X. CONCLUSION AND FUTURE WORK

AI can make technical-debt management more systematic by combining code, repository, natural-language, architectural, and operational evidence. However, debt detection is only one part of risk quantification. A defensible approach must distinguish the probability that debt exists, the one-time principal required to repay it, the future interest generated by retaining it, and the likelihood and impact of adverse events. This paper proposed a multi-source framework that aggregates these outputs into horizon-specific risk exposure and capacity-aware remediation priorities, while preserving uncertainty, provenance, and human review.

The next research step is an empirical, multi-organization evaluation using observed remediation effort and longitudinal incident/change outcomes. Such a study should compare separate prediction heads against strong baselines, test calibration and transfer across debt types, and determine whether risk-adjusted ranking improves engineering allocation relative to principal-only or smell-count prioritization. Further work is also needed on causal attribution, AI-system debt, uncertainty-aware portfolio selection, and safe integration with automated repayment tools. Until those evaluations are available, the proposed equations should be treated as a transparent design hypothesis rather than an established standard.

REFERENCES

[1] V. Lenarduzzi, T. Besker, D. Taibi, A. Martini, and F. A. Fontana, "A systematic literature review on technical debt prioritization: Strategies, processes, factors, and tools," Journal of Systems and Software, vol. 171, Art. no. 110827, 2021, doi: 10.1016/j.jss.2020.110827.

[2] D. Tsoukalas, N. Mittas, A. Chatzigeorgiou, D. D. Kehagias, A. Ampatzoglou, T. Amanatidis, and L. Angelis, "Machine learning for technical debt identification," IEEE Transactions on Software Engineering, vol. 48, no. 12, pp. 4892–4906, 2022, doi: 10.1109/TSE.2021.3129355.

[3] M. T. Baldassarre, V. Lenarduzzi, S. Romano, and N. Saarimäki, "On the diffuseness of technical debt items and accuracy of remediation time when using SonarQube," Information and Software Technology, vol. 128, Art. no. 106377, 2020, doi: 10.1016/j.infsof.2020.106377.

[4] A. Ampatzoglou, N. Mittas, A. A. Tsintzira, A. Ampatzoglou, E.-M. Arvanitou, A. Chatzigeorgiou, P. Avgeriou, and L. Angelis, "Exploring the relation between technical debt principal and interest: An empirical approach," Information and Software Technology, vol. 128, Art. no. 106391, 2020, doi: 10.1016/j.infsof.2020.106391.

[5] P. Ardimento, L. Aversano, M. L. Bernardi, M. Cimitile, and M. Iammarino, "Using deep temporal convolutional networks to just-in-time forecast technical debt principal," Journal of Systems and Software, vol. 194, Art. no. 111481, 2022, doi: 10.1016/j.jss.2022.111481.

[6] Y. Li, M. Soliman, and P. Avgeriou, "Identifying self-admitted technical debt in issue tracking systems using machine learning," Empirical Software Engineering, vol. 27, Art. no. 131, 2022, doi: 10.1007/s10664-022-10128-3.

[7] Y. Kashiwa, R. Nishikawa, Y. Kamei, M. Kondo, E. Shihab, R. Sato, and N. Ubayashi, "An empirical study on self-admitted technical debt in modern code review," Information and Software Technology, vol. 146, Art. no. 106855, 2022, doi: 10.1016/j.infsof.2022.106855.

[8] R. Sharma et al., "Self-admitted technical debt in R: Detection and causes," Automated Software Engineering, vol. 29, Art. no. 53, 2022, doi: 10.1007/s10515-022-00358-6.

[9] E. A. AlOmar, B. Christians, M. Busho, A. H. AlKhalid, A. Ouni, C. Newman, and M. W. Mkaouer, "SATDBailiff—Mining and tracking self-admitted technical debt," Science of Computer Programming, vol. 213, Art. no. 102693, 2022, doi: 10.1016/j.scico.2021.102693.

[10] A. Alhefdhi, H. K. Dam, Y. S. Nugroho, H. Hata, T. Ishio, and A. Ghose, "A framework for conditional statement technical debt identification and description," Automated Software Engineering, vol. 29, Art. no. 60, 2022, doi: 10.1007/s10515-022-00364-8.

[11] A. Melo, R. Fagundes, V. Lenarduzzi, and W. B. Santos, "Identification and measurement of requirements technical debt in software development: A systematic literature review," Journal of Systems and Software, vol. 194, Art. no. 111483, 2022, doi: 10.1016/j.jss.2022.111483.

[12] Y. Li, M. Soliman, and P. Avgeriou, "Automatic identification of self-admitted technical debt from four different sources," Empirical Software Engineering, vol. 28, Art. no. 65, 2023, doi: 10.1007/s10664-023-10297-9.

[13] Y. Li, M. Soliman, P. Avgeriou, and L. Somers, "Self-admitted technical debt in the embedded systems industry: An exploratory case study," IEEE Transactions on Software Engineering, vol. 49, no. 4, pp. 2545–2565, 2023, doi: 10.1109/TSE.2022.3224378.

[14] D. Sas and P. Avgeriou, "An architectural technical debt index based on machine learning and architectural smells," IEEE Transactions on Software Engineering, vol. 49, no. 8, pp. 4169–4195, 2023, doi: 10.1109/TSE.2023.3286179.

[15] Y. Cai and R. Kazman, "Software design analysis and technical debt management based on design rule theory," Information and Software Technology, vol. 164, Art. no. 107322, 2023, doi: 10.1016/j.infsof.2023.107322.

[16] J. P. Biazotto, D. Feitosa, P. Avgeriou, and E. Y. Nakagawa, "Technical debt management automation: State of the art and future perspectives," Information and Software Technology, vol. 167, Art. no. 107375, 2024, doi: 10.1016/j.infsof.2023.107375.

[17] J. Perera, E. Tempero, Y.-C. Tu, and K. Blincoe, "A systematic mapping study exploring quantification approaches to code, design, and architecture technical debt," ACM Transactions on Software Engineering and Methodology, 2024, doi: 10.1145/3675393.

[18] J. Perera, E. Tempero, Y.-C. Tu, and K. Blincoe, "Modelling the quantification of requirements technical debt," Requirements Engineering, vol. 29, pp. 421–458, 2024, doi: 10.1007/s00766-024-00424-3.

[19] A. Alhefdhi, H. K. Dam, and A. Ghose, "Towards automating self-admitted technical debt repayment," Information and Software Technology, vol. 167, Art. no. 107376, 2024, doi: 10.1016/j.infsof.2023.107376.

[20] M. Borg, I. Pruvost, E. Mones, and A. Tornhill, "Increasing, not diminishing: Investigating the returns of highly maintainable code," in Proc. 7th ACM/IEEE International Conference on Technical Debt, 2024, pp. 21–30, doi: 10.1145/3644384.3644471.

[21] M. Siavvas, D. Tsoukalas, M. Jankovic, D. Kehagias, and D. Tzovaras, "Technical debt as an indicator of software security risk: A machine learning approach for software development enterprises," Enterprise Information Systems, vol. 16, no. 5, 2022, doi: 10.1080/17517575.2020.1824017.

[22] M. Robredo, N. Saarimäki, R. Peñaloza, D. Taibi, and V. Lenarduzzi, "Comparing multivariate time series analysis and machine learning performance for technical debt prediction: The SQALE index case," in Proc. 7th ACM/IEEE International Conference on Technical Debt, 2024, pp. 45–46, doi: 10.1145/3644384.3644472.

[23] D. Tsoukalas, N. Mittas, E.-M. Arvanitou, A. Ampatzoglou, A. Chatzigeorgiou, and D. D. Kehagias, "Local and global explainability for technical debt identification," IEEE Transactions on Software Engineering, vol. 50, no. 8, pp. 2110–2123, 2024, doi: 10.1109/TSE.2024.3422427.

[24] G. Recupito, F. Pecorelli, G. Catolino, and V. Lenarduzzi, "Technical debt in AI-enabled systems: On the prevalence, severity, impact, and management strategies for code and architecture," Journal of Systems and Software, vol. 216, Art. no. 112151, 2024, doi: 10.1016/j.jss.2024.112151.

[25] K. Maggi, R. Verdecchia, L. Scommegna, and E. Vicario, "Evolution of code technical debt in microservices architectures," Journal of Systems and Software, vol. 222, Art. no. 112301, 2025, doi: 10.1016/j.jss.2024.112301.

[26] R. A. Halepmollasi and A. Tosun, "Predicting vulnerable code changes using technical debt indicators," IEEE Access, vol. 13, pp. 153896–153917, 2025, doi: 10.1109/ACCESS.2025.3602480.

[27] M. Robredo, N. Saarimäki, M. Esposito, D. Taibi, and R. Peñaloza, "Evaluating time-dependent methods and seasonal effects in code technical debt prediction," Journal of Systems and Software, vol. 230, Art. no. 112545, 2025, doi: 10.1016/j.jss.2025.112545.

[28] E. L. Melin and N. U. Eisty, "Exploring the advances in using machine learning to identify technical debt and self-admitted technical debt," in Proc. 23rd IEEE/ACIS International Conference on Software Engineering Research, Management and Applications (SERA), 2025, pp. 15–22, doi: 10.1109/SERA65747.2025.11154577.

[29] S. Moreschini, E.-M. Arvanitou, E.-P. Kanidou, N. Nikolaidis, R. Su, A. Ampatzoglou, A. Chatzigeorgiou, and V. Lenarduzzi, "The evolution of technical debt from DevOps to generative AI: A multivocal literature review," Journal of Systems and Software, vol. 231, Art. no. 112599, 2026, doi: 10.1016/j.jss.2025.112599.

[30] E. Sutoyo, P. Avgeriou, and A. Capiluppi, "Reducing labeling effort in architecture technical debt detection through active learning and explainable AI," Empirical Software Engineering, vol. 31, Art. no. 183, 2026, doi: 10.1007/s10664-026-10916-1.